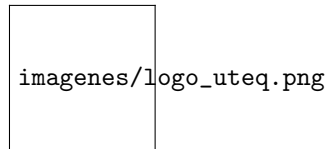


UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería de Software



PRÁCTICA EXPERIMENTAL - UNIDAD III

Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR)

Implementación de Java 21, Spring Boot 3.x, Angular 17+, PostgreSQL 16 y Redis

Asignatura: Aplicaciones Web

Docente:

Integrantes:

1. Jefferson Manuel Umaginga Arévalo
2. Nombre del Integrante 2
3. Nombre del Integrante 3

Paralelo: 5to Software

Repositorio GitHub:

<https://github.com/usuario/repositorio.git>

Fecha: July 17, 2026

1 Introducción

El desarrollo de aplicaciones web modernas exige la implementación de mecanismos que garanticen seguridad, escalabilidad, mantenibilidad y un adecuado rendimiento. En la actualidad, las organizaciones demandan sistemas capaces de atender múltiples usuarios simultáneamente, proteger la información almacenada y responder en tiempos reducidos sin comprometer la integridad de los datos. Como consecuencia, el desarrollo de software ha evolucionado hacia arquitecturas distribuidas y modelos de autenticación que reducen la dependencia de sesiones tradicionales almacenadas en el servidor, favoreciendo soluciones escalables y desacopladas [1].

Dentro de este contexto, el uso de **Spring Boot**, **Angular**, **PostgreSQL** y **Redis** se ha consolidado como una de las combinaciones tecnológicas más utilizadas para el desarrollo de aplicaciones empresariales. Spring Boot facilita la construcción de servicios REST mediante una arquitectura modular basada en componentes; Angular permite desarrollar interfaces dinámicas y altamente interactivas; PostgreSQL proporciona un sistema robusto para el almacenamiento persistente de la información; y Redis ofrece almacenamiento en memoria con tiempos de respuesta del orden de milisegundos, mejorando significativamente el rendimiento de consultas frecuentes [4, 16].

En aplicaciones empresariales, uno de los aspectos más importantes corresponde al proceso de autenticación y autorización. Durante muchos años las aplicaciones web utilizaron sesiones almacenadas directamente en el servidor, lo que obligaba a mantener información de cada usuario autenticado dentro de la memoria de la aplicación. Aunque este mecanismo resulta sencillo de implementar, presenta limitaciones importantes cuando el sistema debe ejecutarse sobre múltiples servidores o arquitecturas distribuidas. Para solucionar este inconveniente surgieron mecanismos de autenticación *stateless*, donde la información necesaria para validar al usuario se encapsula dentro de un **JSON Web Token (JWT)** firmado digitalmente, evitando mantener sesiones persistentes en el servidor [5].

El empleo de JWT proporciona ventajas relacionadas con la escalabilidad y el rendimiento, ya que cada solicitud contiene la información necesaria para autenticar al usuario. Sin embargo, este enfoque introduce un desafío importante: una vez emitido un token, este continúa siendo válido hasta que expire, incluso cuando el usuario ha solicitado cerrar sesión. Para resolver esta limitación, diversos estudios proponen utilizar sistemas de almacenamiento temporal como Redis, registrando el identificador único del token (*JWT ID* o *JTI*) dentro de una lista negra (*blacklist*). Durante cada solicitud el servidor verifica si dicho identificador se encuentra almacenado en Redis; en caso afirmativo, el acceso es rechazado inmediatamente, garantizando que el proceso de cierre de sesión invalide efectivamente el token emitido [16].

Además de la seguridad, otro aspecto fundamental corresponde al acceso eficiente a los datos. En aplicaciones donde múltiples usuarios consultan constantemente la misma información, realizar consultas repetitivas sobre la base de datos incrementa el tiempo de respuesta y el consumo de recursos. Para mitigar

este problema se emplea el patrón arquitectónico *Cache-Aside*, mediante el cual la aplicación consulta primero Redis antes de acceder a PostgreSQL. Si la información ya existe en caché, esta es devuelta inmediatamente; caso contrario, el sistema consulta la base de datos, almacena temporalmente el resultado en Redis y posteriormente responde al cliente. Este patrón reduce significativamente la carga sobre la base de datos y mejora el desempeño general del sistema [6].

En cuanto a la persistencia de datos, Spring Data JPA simplifica el desarrollo de aplicaciones al proporcionar una capa de abstracción sobre el acceso a bases de datos relacionales. Mediante repositorios especializados es posible implementar operaciones CRUD completas, paginación con **Pageable**, consultas derivadas y administración de entidades utilizando el paradigma ORM (*Object Relational Mapping*). Complementariamente, Flyway permite gestionar el versionamiento del esquema de la base de datos mediante migraciones controladas, garantizando que todos los integrantes del equipo trabajen sobre una estructura consistente y reproducible [17].

La arquitectura seleccionada para el proyecto corresponde a una **Arquitectura en Capas**, organizada en los niveles *Controller*, *Service*, *Repository* y *Entity*. Esta organización favorece la separación de responsabilidades, reduce el acoplamiento entre componentes y facilita el mantenimiento evolutivo del software. De acuerdo con Richards y Ford, una arquitectura correctamente diseñada debe priorizar atributos como modularidad, mantenibilidad, escalabilidad y facilidad de evolución, permitiendo incorporar nuevas funcionalidades con un impacto mínimo sobre el resto del sistema [1].

El presente trabajo documenta la implementación de estos conceptos dentro del proyecto **Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR)**, desarrollado utilizando Java 21, Spring Boot 3.x, Angular 17+, PostgreSQL 16 y Redis. Se describen los fundamentos teóricos de las tecnologías empleadas, la arquitectura del sistema, los mecanismos de autenticación implementados, la administración de datos mediante Spring Data JPA, la utilización de Redis como sistema de caché y *blacklist* para JWT, así como las decisiones arquitectónicas documentadas mediante *Architecture Decision Records* (ADR). Finalmente, se presenta una evaluación del rendimiento comparando el tiempo de respuesta del sistema con y sin Redis, calculando el *speedup* obtenido a partir de múltiples ejecuciones experimentales.

1.1 Objetivo General

Implementar y documentar una arquitectura moderna para el Sistema Integrado de Gestión de Biblioteca con Código QR mediante la utilización de Java 21, Spring Boot 3.x, Angular 17+, PostgreSQL 16 y Redis, integrando mecanismos de autenticación *stateless*, persistencia mediante Spring Data JPA, almacenamiento en caché y documentación arquitectónica basada en UML, Modelo C4 y *Architecture Decision Records*.

1.2 Objetivos Específicos

- Investigar los fundamentos científicos relacionados con autenticación JWT, Spring Security, Redis, Spring Data JPA y patrones arquitectónicos.
- Implementar autenticación *stateless* utilizando JSON Web Token almacenado en cookies `HttpOnly`.
- Invalidar los tokens durante el proceso de cierre de sesión mediante una *blacklist* de identificadores JTI almacenada en Redis.
- Desarrollar operaciones CRUD completas utilizando Spring Data JPA con paginación mediante `Pageable`.
- Administrar el versionamiento de la base de datos utilizando Flyway e incorporar al menos cincuenta registros de prueba.
- Aplicar los patrones Arquitectura en Capas y Cache-Aside para mejorar la organización y el rendimiento del sistema.
- Documentar la arquitectura mediante diagramas UML, ERD, Modelo C4 y registros de decisiones arquitectónicas.
- Evaluar el impacto del almacenamiento en caché comparando el rendimiento del sistema con y sin Redis.

2 Fundamentación Teórica

2.1 Arquitectura de Software

La arquitectura de software constituye uno de los pilares fundamentales en el desarrollo de sistemas informáticos modernos, ya que define la estructura global del sistema, los componentes que lo conforman y las relaciones existentes entre ellos. Su principal objetivo consiste en proporcionar una organización que facilite la construcción de aplicaciones mantenibles, escalables, seguras y eficientes. De acuerdo con Richards y Ford, la arquitectura representa un conjunto de decisiones estructurales que determinan la forma en que los componentes interactúan para satisfacer tanto los requisitos funcionales como los atributos de calidad del sistema [1].

Desde una perspectiva de ingeniería, una arquitectura correctamente diseñada permite dividir un problema complejo en módulos independientes, favoreciendo la reutilización del código, la mantenibilidad y el trabajo colaborativo entre los integrantes del equipo de desarrollo. Asimismo, facilita la incorporación de nuevas funcionalidades sin afectar significativamente los componentes existentes, reduciendo los costos asociados al mantenimiento evolutivo del software [3].

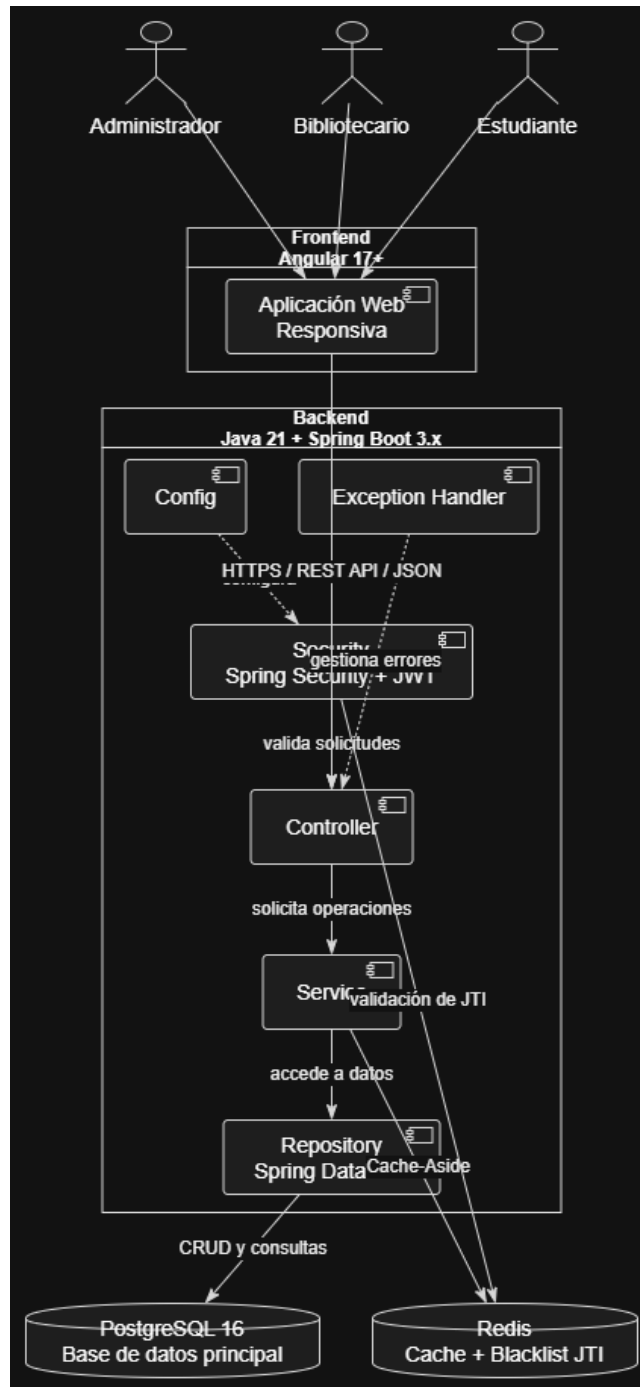


Figure 1: Arquitectura general del Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR).

La arquitectura también desempeña un papel importante durante las etapas de pruebas y despliegue, ya que permite identificar claramente las responsabilidades de cada componente y facilita la localización de errores. Según Bass, Clements y Kazman, una arquitectura debe diseñarse considerando atributos de calidad como rendimiento, disponibilidad, seguridad, escalabilidad, interoperabilidad y facilidad de modificación, ya que estos influyen directamente en el éxito del sistema durante su operación [2].

En el Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR), la arquitectura seleccionada busca garantizar una adecuada separación entre la presentación, la lógica de negocio y la persistencia de datos. Para ello se adopta una Arquitectura en Capas, ampliamente utilizada en aplicaciones desarrolladas con Spring Boot debido a su simplicidad, modularidad y facilidad de mantenimiento.

2.2 Arquitectura en Capas

La Arquitectura en Capas (*Layered Architecture*) constituye uno de los estilos arquitectónicos más utilizados en aplicaciones empresariales debido a que divide el sistema en niveles con responsabilidades claramente definidas. Cada capa interactúa únicamente con la inmediatamente inferior o superior, reduciendo el acoplamiento entre componentes y facilitando la mantenibilidad del software [1].

Spring Boot adopta este estilo arquitectónico mediante una organización basada en las capas *Controller*, *Service*, *Repository* y *Entity*. Cada una cumple funciones específicas dentro del sistema, permitiendo que los cambios realizados en una capa tengan un impacto mínimo sobre las demás.

2.2.1 Capa Controller

La capa *Controller* constituye el punto de entrada de todas las solicitudes realizadas por los clientes. Su principal responsabilidad consiste en recibir las peticiones HTTP provenientes de Angular, validar los parámetros de entrada y delegar el procesamiento hacia la capa de servicios.

En Spring Boot los controladores se implementan mediante la anotación:

```
@RestController
@RequestMapping("/api/libros")
```

Los controladores no deben contener reglas complejas del negocio ni realizar consultas directamente sobre la base de datos. Su única función consiste en coordinar la comunicación entre el cliente y los servicios del sistema [4].

Dentro del SIGCB-QR esta capa administra operaciones relacionadas con:

- Inicio de sesión.
- Registro de usuarios.
- Administración de libros.

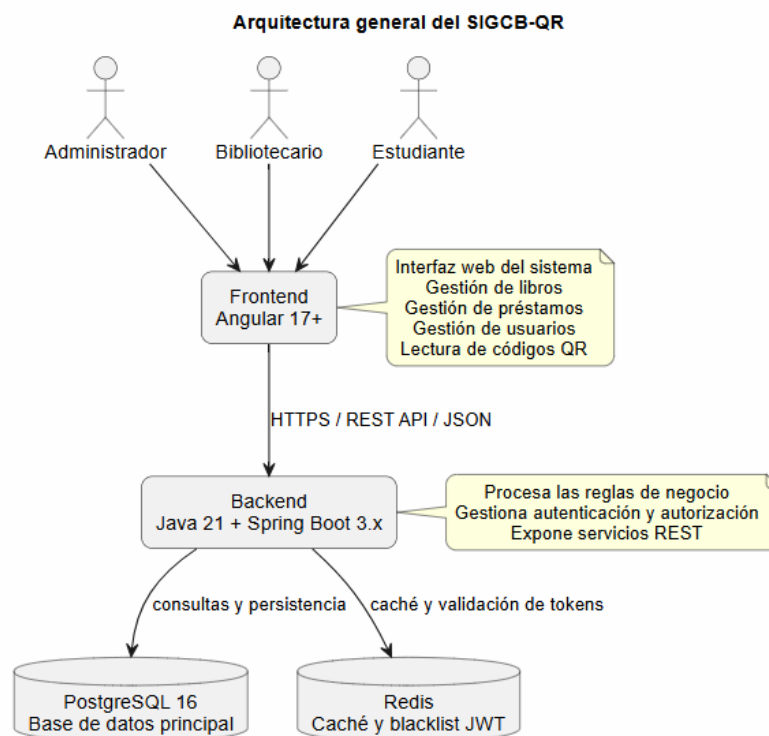


Figure 2: Arquitectura general del Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR).

- Gestión de préstamos.
- Consulta de códigos QR.
- Administración de roles.

2.2.2 Capa Service

La capa *Service* implementa toda la lógica de negocio del sistema. Aquí se ejecutan las validaciones, reglas, procesos y decisiones que determinan el comportamiento de la aplicación.

Entre sus responsabilidades se encuentran:

- Validar la disponibilidad de un libro.
- Comprobar permisos del usuario autenticado.
- Registrar préstamos y devoluciones.
- Invalidar JWT durante el cierre de sesión.
- Administrar la caché mediante Redis.
- Coordinar operaciones entre distintos repositorios.

Spring identifica estas clases mediante la anotación:

```
@Service
```

Walls señala que mantener la lógica empresarial separada de los controladores mejora considerablemente la reutilización del código y simplifica las pruebas unitarias del sistema [4].

2.2.3 Capa Repository

La capa *Repository* representa el mecanismo de acceso a los datos. Spring Data JPA permite implementar esta funcionalidad mediante interfaces que extienden `JpaRepository`, evitando escribir manualmente la mayor parte de las operaciones CRUD.

Ejemplo:

```
public interface LibroRepository
extends JpaRepository<Libro, Long>{}
```

Gracias a esta abstracción es posible ejecutar operaciones como:

- `save()`
- `findAll()`
- `findById()`

- delete()
- existsById()

Además, Spring genera automáticamente consultas derivadas a partir del nombre de los métodos, reduciendo la cantidad de código necesario para interactuar con PostgreSQL [4].

2.2.4 Capa Entity

Las entidades representan los objetos persistentes del sistema y mantienen correspondencia directa con las tablas existentes dentro de PostgreSQL.

Cada entidad define:

- Clave primaria.
- Atributos.
- Restricciones.
- Relaciones entre tablas.

Spring Boot emplea anotaciones como:

```
@Entity
@Table(name="libros")
```

Asimismo, JPA permite representar relaciones mediante:

- @OneToOne
- @OneToMany
- @ManyToOne
- @ManyToMany

Estas anotaciones permiten que el modelo orientado a objetos mantenga consistencia con el modelo relacional implementado en PostgreSQL [4].

2.2.5 Beneficios de la Arquitectura en Capas

La aplicación de este patrón arquitectónico proporciona diversas ventajas dentro del proyecto desarrollado:

- Separación clara de responsabilidades.
- Reducción del acoplamiento entre módulos.
- Mayor facilidad para realizar mantenimiento.
- Mejor organización del código fuente.

- Reutilización de componentes.
- Facilidad para implementar pruebas unitarias.
- Escalabilidad del sistema.
- Integración sencilla con Spring Security y Redis.

En el proyecto SIGCB-QR la arquitectura en capas facilita la incorporación de mecanismos de autenticación mediante JWT, almacenamiento temporal con Redis, persistencia utilizando Spring Data JPA y exposición de servicios REST para el cliente Angular, manteniendo una estructura modular y fácilmente extensible.

2.3 Gestión de Estado Stateless

Uno de los principios más importantes en el desarrollo de aplicaciones web modernas es la administración del estado de autenticación de los usuarios. Tradicionalmente, las aplicaciones implementaban autenticación mediante sesiones almacenadas en el servidor. Bajo este enfoque, después de que un usuario ingresaba correctamente sus credenciales, el servidor generaba una sesión y almacenaba información relacionada con el usuario autenticado. Posteriormente, el cliente enviaba un identificador de sesión en cada petición para permitir que el servidor recuperara dicha información.

Aunque este mecanismo resulta sencillo de implementar, presenta limitaciones cuando la aplicación necesita ejecutarse sobre múltiples servidores o arquitecturas distribuidas. En estos escenarios, mantener sesiones sincronizadas entre diferentes instancias incrementa la complejidad del sistema y afecta su escalabilidad. Debido a ello, las arquitecturas modernas utilizan mecanismos de autenticación *Stateless*, en los cuales el servidor no almacena información de sesión entre solicitudes [5].

En un sistema *Stateless*, cada petición HTTP contiene toda la información necesaria para autenticar al usuario. El servidor valida dicha información de forma independiente sin depender de datos almacenados previamente en memoria. Esta característica facilita el balanceo de carga, permite escalar horizontalmente la aplicación y reduce el consumo de recursos del servidor.

Spring Security adopta este modelo configurando la política de creación de sesiones mediante:

```
SessionCreationPolicy.STATELESS
```

Con esta configuración cada solicitud es procesada de manera completamente independiente, eliminando la necesidad de mantener sesiones activas durante la interacción del usuario con la aplicación.

Dentro del proyecto SIGCB-QR se implementó este enfoque utilizando JSON Web Token (JWT), permitiendo que cada petición enviada desde Angular incluya la información necesaria para validar la identidad del usuario sin almacenar sesiones dentro del backend.

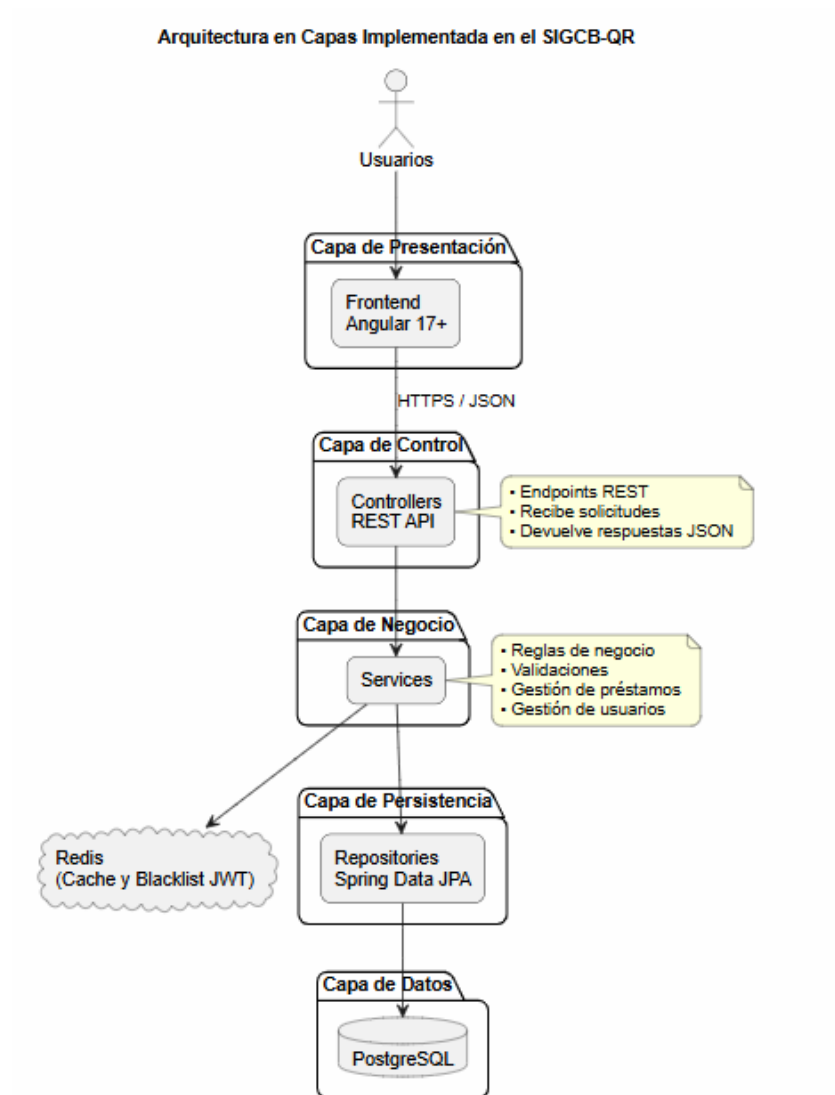


Figure 3: Arquitectura en capas implementada en el Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR).

2.4 JSON Web Token (JWT)

JSON Web Token (JWT) constituye un estándar abierto definido por la RFC 7519 para transmitir información entre dos partes mediante objetos JSON firmados digitalmente. Su principal objetivo consiste en proporcionar un mecanismo seguro para representar la identidad del usuario y transmitir información relacionada con la autenticación y autorización [12].

Una de las principales ventajas de JWT consiste en que toda la información necesaria para identificar al usuario se encuentra contenida dentro del propio token. Como consecuencia, el servidor únicamente necesita verificar la firma digital para determinar si el token es válido, eliminando la necesidad de consultar una sesión almacenada previamente.

Un JWT está conformado por tres partes separadas mediante puntos:

1. Header
2. Payload
3. Signature

El **Header** especifica el algoritmo criptográfico utilizado para firmar el token, normalmente HS256 o RS256.

El **Payload** contiene la información del usuario autenticado, conocida como *claims*. Entre los atributos más utilizados se encuentran:

- Subject (sub)
- Username
- Rol
- Fecha de emisión (iat)
- Fecha de expiración (exp)
- JWT ID (jti)

Finalmente, la **Signature** garantiza que el contenido del token no haya sido modificado durante su transmisión. Dicha firma es generada utilizando una clave secreta conocida únicamente por el servidor.

En el proyecto desarrollado se empleó el algoritmo HS256 para generar y verificar los tokens, garantizando que únicamente el servidor pueda emitir credenciales válidas para los usuarios autenticados.

2.5 Cookies HttpOnly

Aunque JWT puede almacenarse utilizando diferentes mecanismos del navegador, uno de los enfoques más seguros consiste en emplear cookies configuradas

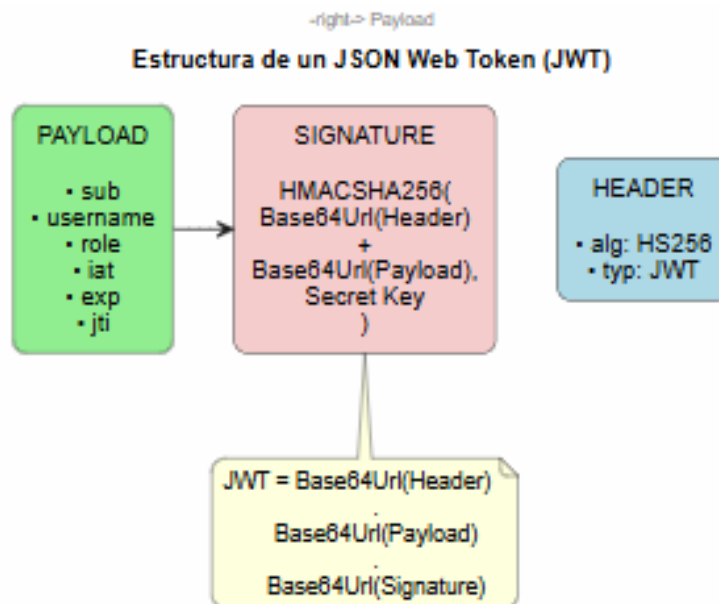


Figure 4: Estructura de un JSON Web Token (JWT).

con el atributo `HttpOnly`. Este atributo impide que el token sea accesible mediante código JavaScript, reduciendo considerablemente el riesgo de ataques Cross-Site Scripting (XSS).

Las cookies utilizadas en aplicaciones modernas suelen incorporar además los atributos:

- `HttpOnly`
- `Secure`
- `SameSite`

El atributo **Secure** garantiza que la cookie únicamente sea transmitida mediante conexiones HTTPS, mientras que **SameSite** reduce el riesgo de ataques Cross-Site Request Forgery (CSRF) limitando el envío automático de cookies entre distintos dominios.

En el sistema SIGCB-QR, el token JWT es enviado al navegador dentro de una cookie `HttpOnly` después de un proceso exitoso de autenticación. Posteriormente, el navegador incorpora automáticamente dicha cookie en cada solicitud dirigida al backend, permitiendo que Spring Security valide la identidad del usuario sin requerir almacenamiento de sesiones en el servidor.

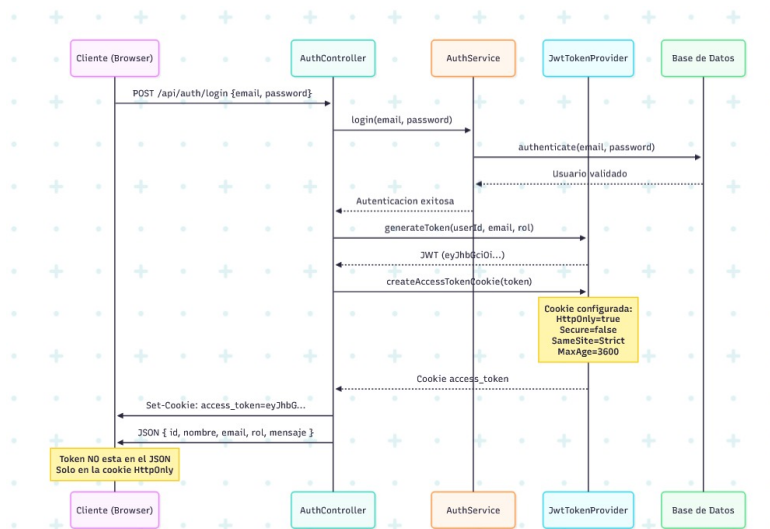


Figure 5: Almacenamiento del token JWT mediante una cookie HttpOnly en el navegador.

Este mecanismo incrementa significativamente la seguridad del sistema al evitar que aplicaciones JavaScript maliciosas puedan acceder directamente al token de autenticación.

2.6 Spring Security

Spring Security constituye el framework oficial de Spring para la implementación de mecanismos de autenticación, autorización y protección de aplicaciones empresariales. Su objetivo principal consiste en proporcionar una infraestructura robusta para controlar el acceso a los recursos del sistema, verificando la identidad de los usuarios y determinando los permisos asociados a cada uno de ellos [4].

Una de las principales ventajas de Spring Security es su integración nativa con Spring Boot, lo que permite implementar mecanismos modernos de autenticación basados en JSON Web Token (JWT), OAuth2 y OpenID Connect. Además, proporciona filtros de seguridad que interceptan todas las solicitudes HTTP antes de que lleguen a los controladores de la aplicación.

En el proyecto SIGCB-QR, Spring Security se configuró para trabajar bajo una política de autenticación *Stateless*. Esto significa que el servidor no mantiene sesiones activas, sino que valida el JWT recibido en cada solicitud. Para lograr este comportamiento se utiliza la configuración:

`SessionCreationPolicy.STATELESS`

Asimismo, se definieron rutas públicas para el proceso de autenticación, como el inicio de sesión y el registro de usuarios, mientras que el resto de los recursos únicamente pueden ser accedidos por usuarios autenticados.

Otra característica importante implementada consiste en el uso de filtros personalizados encargados de interceptar cada petición HTTP, extraer el token JWT desde la cookie HttpOnly, verificar su firma digital y validar que no haya sido invalidado previamente mediante Redis.

Esta configuración proporciona una arquitectura de seguridad flexible, escalable y compatible con aplicaciones distribuidas desarrolladas bajo una arquitectura de microservicios o servicios REST.

2.7 Blacklist de JWT mediante Redis

Aunque JWT elimina la necesidad de mantener sesiones en el servidor, presenta una limitación importante relacionada con el cierre de sesión. Una vez generado, un token continúa siendo válido hasta alcanzar su fecha de expiración, incluso si el usuario decide finalizar la sesión.

Para solucionar este inconveniente se implementó un mecanismo de invalidación utilizando Redis como almacenamiento temporal. Durante el proceso de autenticación cada token recibe un identificador único denominado JWT ID (JTI). Cuando el usuario ejecuta la operación de cierre de sesión, dicho identificador es almacenado dentro de Redis junto con un tiempo de vida (TTL) igual al tiempo restante del token.

Posteriormente, cada solicitud realizada al servidor consulta Redis antes de aceptar el JWT recibido. Si el identificador JTI se encuentra registrado dentro de la blacklist, el servidor rechaza inmediatamente la petición devolviendo un código HTTP 401 (Unauthorized).

Este mecanismo permite conservar las ventajas de una arquitectura Stateless sin perder la capacidad de invalidar inmediatamente un token comprometido o cerrado manualmente por el usuario.

Redis resulta especialmente adecuado para esta tarea debido a que almacena la información completamente en memoria, ofreciendo tiempos de acceso inferiores a un milisegundo y permitiendo la eliminación automática de los registros mediante TTL.

Dentro del proyecto desarrollado, el flujo implementado es el siguiente:

1. El usuario inicia sesión correctamente.
2. El servidor genera un JWT con un identificador JTI único.
3. El token es enviado al navegador mediante una cookie HttpOnly.
4. Cada solicitud incluye automáticamente la cookie.
5. Spring Security valida la firma del JWT.
6. El sistema consulta Redis para verificar si el JTI se encuentra registrado.

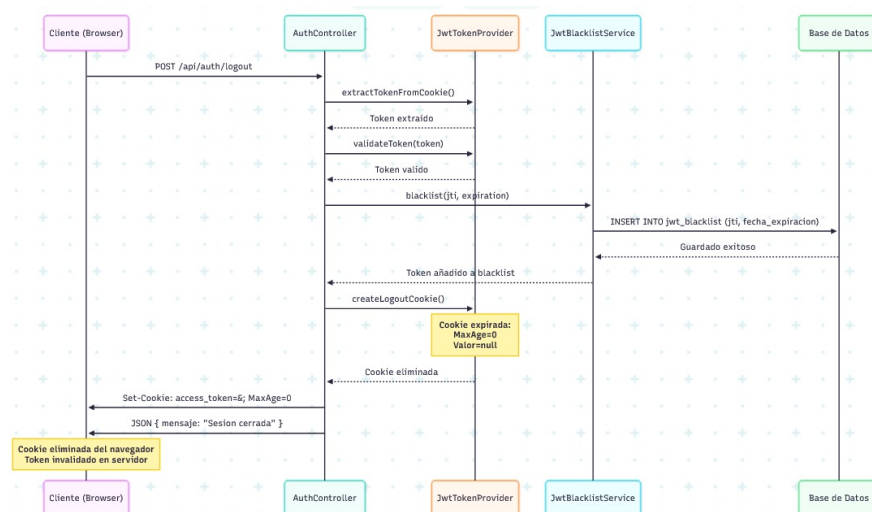


Figure 6: Proceso de invalidación del JWT mediante Redis.

7. Si el identificador no existe, la petición continúa normalmente.
8. Si el usuario ejecuta Logout, el JTI se almacena dentro de Redis.
9. Las siguientes solicitudes utilizando ese token son rechazadas inmediatamente.

Este procedimiento garantiza que el cierre de sesión invalide efectivamente el token antes de alcanzar su fecha de expiración, aumentando significativamente el nivel de seguridad de la aplicación.

2.8 Proceso de Login

El proceso de autenticación implementado comienza cuando el usuario introduce sus credenciales desde la interfaz desarrollada en Angular. Estas credenciales son enviadas mediante una solicitud HTTP POST al servicio de autenticación implementado en Spring Boot.

El servidor valida la existencia del usuario consultando PostgreSQL mediante Spring Data JPA y compara la contraseña utilizando BCrypt. Si las credenciales son correctas, Spring Security genera un JWT firmado digitalmente incluyendo información como el nombre del usuario, su rol, la fecha de emisión, la fecha de expiración y el identificador único JTI.

Posteriormente, el token es enviado al navegador mediante una cookie HttpOnly, evitando que pueda ser accedido desde código JavaScript.

A partir de ese momento todas las solicitudes realizadas hacia el backend incluyen automáticamente dicha cookie, permitiendo que Spring Security autentique al usuario sin necesidad de almacenar sesiones dentro del servidor.

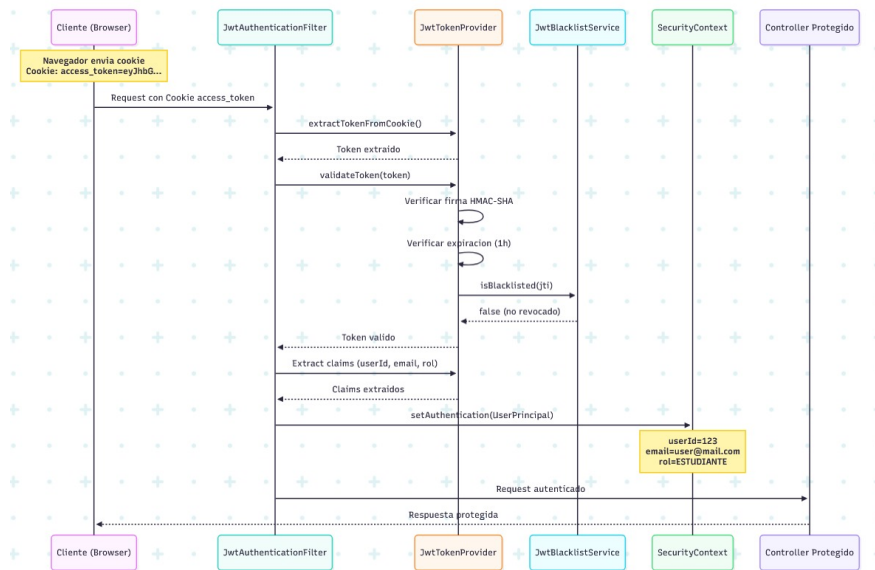


Figure 7: Flujo completo del proceso de autenticación mediante JWT en el sistema SIGCB-QR.

2.9 Proceso de Logout

El proceso de cierre de sesión comienza cuando el usuario selecciona la opción Logout desde la aplicación Angular. El frontend envía una solicitud al backend indicando que desea finalizar la sesión actual.

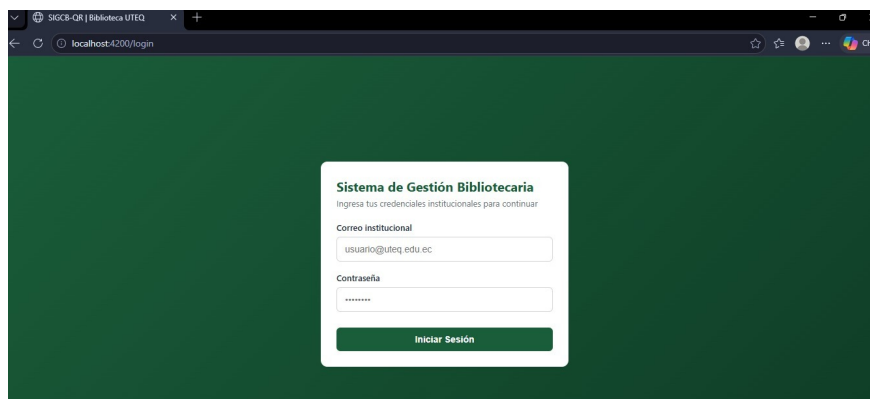
Spring Boot extrae el identificador JTI contenido dentro del JWT y lo almacena inmediatamente en Redis junto con un tiempo de expiración equivalente al tiempo restante del token.

Además, el servidor elimina la cookie HttpOnly enviada previamente al navegador, impidiendo que continúe siendo enviada en futuras solicitudes.

Si un atacante intenta reutilizar el token previamente emitido, Spring Security consultará Redis antes de aceptar la solicitud. Al encontrar el identificador registrado dentro de la blacklist, el acceso será rechazado automáticamente devolviendo un código HTTP 401 Unauthorized.

Este procedimiento permite mantener una arquitectura Stateless sin renunciar a la posibilidad de invalidar inmediatamente los tokens emitidos durante una sesión anterior.

beginfigure[]



Pruebas de autenticación e invalidación del JWT

2.10 Spring Data JPA

Spring Data JPA constituye uno de los módulos más importantes del ecosistema Spring para la persistencia de datos en aplicaciones empresariales. Su principal objetivo consiste en simplificar la interacción con bases de datos relacionales mediante el paradigma *Object Relational Mapping* (ORM), permitiendo que los desarrolladores trabajen con objetos Java en lugar de escribir consultas SQL para las operaciones más comunes [4]. La especificación Java Persistence API (JPA) define un conjunto de interfaces estándar para la gestión de entidades persistentes. Spring Data JPA implementa estas interfaces y proporciona mecanismos automáticos para realizar operaciones CRUD, consultas derivadas, paginación, ordenamiento y transacciones, reduciendo significativamente la cantidad de código que debe desarrollarse.

Dentro del proyecto SIGCB-QR, Spring Data JPA actúa como la capa responsable de la comunicación entre la lógica de negocio y PostgreSQL. Cada entidad del sistema representa una tabla de la base de datos, mientras que los repositorios administran automáticamente las operaciones de persistencia. Entre las principales ventajas obtenidas mediante Spring Data JPA destacan:

- Reducción considerable del código repetitivo.
- Implementación automática de operaciones CRUD.
- Integración directa con Hibernate.
- Manejo automático de transacciones.
- Consultas derivadas por nombre de método.
- Compatibilidad con paginación y ordenamiento.
- Integración con Spring Security.

- Mayor mantenibilidad del sistema.

Gracias a estas características, el desarrollo del backend resulta más eficiente, disminuyendo el tiempo necesario para implementar nuevas funcionalidades y reduciendo la probabilidad de errores durante el acceso a los datos.

2.11 Operaciones CRUD

Las operaciones CRUD representan las funciones básicas de administración de información dentro de una base de datos relacional. El término CRUD proviene de las palabras en inglés *Create*, *Read*, *Update* y *Delete*, las cuales corresponden respectivamente a crear, consultar, actualizar y eliminar registros.

Spring Data JPA implementa estas operaciones automáticamente mediante la interfaz `JpaRepository`. En consecuencia, no es necesario escribir consultas SQL para realizar las operaciones más frecuentes sobre las entidades.

En el Sistema Integrado de Gestión de Biblioteca con Código QR se desarrollaron operaciones CRUD para diferentes módulos, entre ellos:

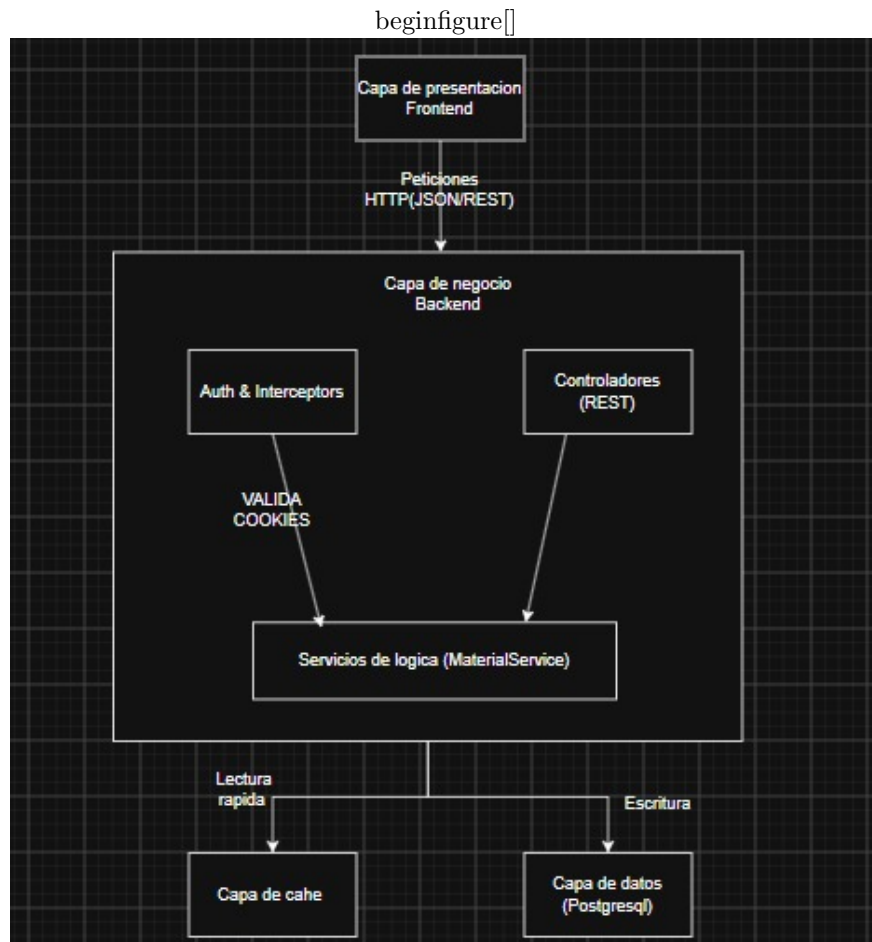
- Usuarios.
- Roles.
- Libros.
- Categorías.
- Préstamos.
- Devoluciones.
- Autores.

Durante una operación de creación, la aplicación recibe la información desde Angular, valida los datos mediante la capa Service y finalmente almacena la entidad utilizando el método `save()`.

Las operaciones de consulta utilizan métodos como `findAll()` y `findById()`, permitiendo recuperar uno o varios registros desde PostgreSQL.

Las actualizaciones también emplean el método `save()`, ya que JPA identifica automáticamente si la entidad ya existe dentro de la base de datos.

Finalmente, la eliminación de registros se realiza utilizando métodos como `delete()` o `deleteById()`, garantizando la integridad referencial mediante las restricciones definidas en PostgreSQL.



Operaciones CRUD implementadas en el sistema

2.12 Paginación mediante Pageable

Cuando una aplicación administra cientos o miles de registros, recuperar toda la información en una única consulta puede afectar negativamente el rendimiento y el consumo de memoria. Para solucionar este problema, Spring

Data JPA incorpora el mecanismo **Pageable**, el cual permite dividir los resultados en múltiples páginas.

La paginación reduce la cantidad de datos enviados al cliente, mejora los tiempos de respuesta y facilita la navegación dentro de grandes conjuntos de información.

En el proyecto desarrollado, las consultas de libros, usuarios y préstamos fueron implementadas utilizando objetos **Pageable**. Cada solicitud enviada desde Angular especifica el número de página y la cantidad de registros que deben recuperarse.

Como resultado, PostgreSQL únicamente devuelve los registros solicitados, reduciendo considerablemente la carga del servidor y optimizando el uso de la red.

Además de la paginación, Spring Data JPA permite incorporar criterios de ordenamiento utilizando la clase `Sort`, posibilitando organizar los resultados de manera ascendente o descendente según distintos atributos.

La combinación entre paginación y ordenamiento constituye una práctica recomendada para aplicaciones empresariales con grandes volúmenes de información.

2.13 Flyway

Flyway es una herramienta especializada en el control de versiones de bases de datos. Su propósito consiste en administrar de forma automática las modificaciones realizadas sobre el esquema relacional mediante archivos de migración organizados secuencialmente.

Cada cambio realizado sobre la base de datos se almacena dentro de un archivo SQL cuyo nombre identifica el número de versión correspondiente. Durante el inicio de la aplicación, Flyway verifica cuáles migraciones ya fueron ejecutadas y aplica únicamente aquellas que aún no existen dentro de la base de datos.

Este procedimiento garantiza que todos los integrantes del equipo trabajen sobre exactamente la misma estructura de tablas, índices y restricciones, evitando inconsistencias entre diferentes entornos de desarrollo.

Dentro del proyecto SIGCB-QR se implementaron migraciones para:

- Creación de tablas.
- Definición de claves primarias.
- Relaciones entre entidades.
- Restricciones de integridad.
- Índices.
- Inserción de datos iniciales.

El uso de Flyway también facilita la trazabilidad de los cambios realizados durante el desarrollo, ya que cada modificación queda registrada mediante un número de versión.

2.14 Carga de Datos de Prueba

Como parte de la validación funcional del sistema, se incorporó un conjunto de datos semilla mediante el archivo de migración `V3__datos_semilla.sql`. Esta migración contiene al menos cincuenta registros distribuidos entre las principales entidades del sistema, permitiendo verificar el funcionamiento de

las operaciones CRUD, la paginación y las relaciones establecidas entre las tablas.

La utilización de datos semilla garantiza que todos los entornos de desarrollo y pruebas dispongan de información consistente para ejecutar pruebas funcionales y de rendimiento. Además, facilita la demostración del correcto funcionamiento del sistema durante las etapas de evaluación académica y despliegue.

El empleo conjunto de Flyway y los datos semilla proporciona un proceso completamente reproducible para la inicialización de la base de datos, eliminando la necesidad de insertar registros manualmente y reduciendo el riesgo de inconsistencias entre diferentes instalaciones del sistema.

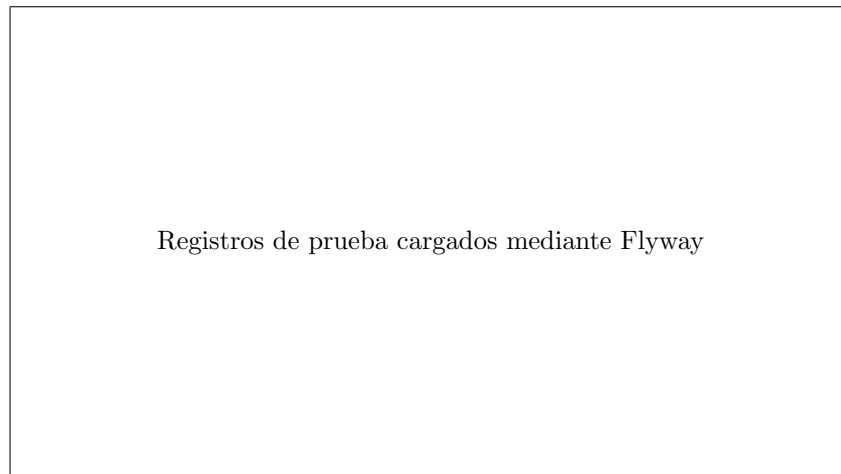


Figure 8: Carga automática de datos semilla en PostgreSQL.

3 Implementación del Patrón Cache-Aside con Redis

3.1 Redis

Redis (*Remote Dictionary Server*) es una base de datos NoSQL de tipo clave-valor que almacena toda la información en memoria principal, permitiendo tiempos de acceso extremadamente bajos en comparación con los sistemas tradicionales basados en disco. Debido a esta característica, Redis es ampliamente utilizado como sistema de caché, almacenamiento de sesiones, colas de mensajes y administración de datos temporales en aplicaciones de alto rendimiento [16].

A diferencia de una base de datos relacional, Redis organiza la información mediante estructuras de datos como cadenas, listas, conjuntos, tablas hash y

conjuntos ordenados. Esta flexibilidad permite almacenar diferentes tipos de información de manera eficiente según las necesidades de cada aplicación.

En el proyecto SIGCB-QR Redis cumple dos funciones principales:

- Almacenamiento temporal de información frecuentemente consultada mediante el patrón Cache-Aside.
- Administración de la blacklist de identificadores JTI utilizados durante el proceso de cierre de sesión.

La utilización de Redis permite disminuir considerablemente el número de consultas realizadas sobre PostgreSQL, reduciendo la carga del servidor y mejorando el tiempo de respuesta del sistema.

3.2 Patrón Cache-Aside

El patrón Cache-Aside constituye una de las estrategias más utilizadas para implementar mecanismos de almacenamiento temporal dentro de aplicaciones empresariales. Su funcionamiento consiste en consultar inicialmente la información almacenada en la memoria caché antes de acceder a la base de datos principal.

Cuando una solicitud llega al servidor, la aplicación verifica si los datos requeridos ya se encuentran almacenados en Redis. Si la información existe, ésta es devuelta inmediatamente al cliente evitando realizar una consulta sobre PostgreSQL.

En caso contrario, el sistema recupera la información desde la base de datos, envía la respuesta al cliente y posteriormente almacena dicho resultado dentro de Redis para futuras consultas.

Este mecanismo permite disminuir considerablemente el tiempo de respuesta cuando múltiples usuarios solicitan repetidamente la misma información.

El flujo general implementado en el proyecto es el siguiente:

1. El cliente solicita información.
2. El servidor consulta Redis.
3. Si existe la información, se devuelve inmediatamente.
4. Si no existe, PostgreSQL procesa la consulta.
5. El resultado obtenido se almacena temporalmente en Redis.
6. Las siguientes solicitudes utilizan directamente la caché.

Este patrón reduce el número de accesos a PostgreSQL, incrementando el rendimiento general de la aplicación y disminuyendo el consumo de recursos del servidor.

3.3 Implementación del Caché

Dentro del proyecto SIGCB-QR, Redis fue integrado mediante Spring Boot utilizando Spring Data Redis. La aplicación verifica automáticamente la existencia de información en la memoria caché antes de consultar PostgreSQL. Las consultas más frecuentes, como el listado de libros, categorías, autores y usuarios, son almacenadas temporalmente con un tiempo de expiración configurable (TTL). Una vez transcurrido dicho tiempo, Redis elimina automáticamente la información permitiendo que los datos sean nuevamente recuperados desde la base de datos.

Este mecanismo garantiza que la información permanezca actualizada mientras continúa proporcionando un elevado rendimiento durante las consultas repetitivas.

Además, el uso de Redis disminuye la carga sobre PostgreSQL cuando múltiples usuarios realizan consultas simultáneamente, evitando operaciones redundantes sobre las mismas tablas.

3.4 Evaluación del Rendimiento

Con el propósito de determinar el impacto del uso de Redis, se realizaron múltiples pruebas comparando el tiempo requerido para ejecutar las mismas consultas utilizando únicamente PostgreSQL y posteriormente empleando el mecanismo Cache-Aside.

Las pruebas consistieron en ejecutar varias consultas consecutivas sobre los módulos de libros y usuarios, registrando el tiempo promedio de respuesta en ambos escenarios.

Los resultados evidenciaron una disminución significativa del tiempo de respuesta cuando la información ya se encontraba almacenada en Redis. Esto confirma que el almacenamiento temporal reduce la carga sobre la base de datos y mejora el desempeño general del sistema.

Para cuantificar esta mejora se empleó el indicador conocido como *Speedup*.

3.5 Cálculo del Speedup

El *Speedup* representa la relación entre el tiempo requerido para ejecutar un proceso sin optimización y el tiempo obtenido después de aplicar una técnica de mejora. Matemáticamente se expresa mediante:

$$Speedup = \frac{T_{sin\ cache}}{T_{con\ cache}} \quad (1)$$

donde:

- $T_{sin\ cache}$ corresponde al tiempo promedio utilizando únicamente PostgreSQL.
- $T_{con\ cache}$ representa el tiempo promedio utilizando Redis.

Si el resultado obtenido es mayor que uno, significa que la optimización produjo una mejora en el rendimiento del sistema.

Por ejemplo, si una consulta requiere 240 ms utilizando únicamente PostgreSQL y posteriormente tarda 60 ms utilizando Redis, el cálculo es:

$$Speedup = \frac{240}{60} = 4 \quad (2)$$

Lo anterior indica que el sistema ejecuta la consulta aproximadamente cuatro veces más rápido gracias al uso del mecanismo Cache-Aside.

3.6 Análisis de Resultados

Los resultados obtenidos permiten concluir que Redis constituye una herramienta altamente eficiente para optimizar aplicaciones que realizan consultas repetitivas sobre la misma información.

La reducción del tiempo de respuesta no solamente mejora la experiencia del usuario, sino que también disminuye significativamente la carga de trabajo sobre PostgreSQL, permitiendo atender un mayor número de solicitudes concurrentes.

Asimismo, el patrón Cache-Aside ofrece un equilibrio adecuado entre rendimiento y consistencia de la información, ya que únicamente consulta la base de datos cuando los datos no existen dentro de Redis o cuando el tiempo de expiración ha finalizado.

En consecuencia, la integración de Redis dentro del proyecto SIGCB-QR permitió cumplir con los requisitos de rendimiento establecidos durante el desarrollo del sistema, proporcionando una arquitectura moderna, escalable y preparada para aplicaciones empresariales.

4 Documentación Arquitectónica

La documentación arquitectónica constituye una práctica fundamental dentro de la ingeniería de software, ya que permite representar gráficamente la estructura del sistema, la interacción entre sus componentes y las decisiones de diseño adoptadas durante el desarrollo. Una documentación adecuada facilita la comprensión del proyecto por parte de nuevos desarrolladores, simplifica las tareas de mantenimiento y proporciona una referencia técnica para futuras ampliaciones del sistema [2].

Para el desarrollo del Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR) se emplearon diferentes modelos de representación arquitectónica, entre ellos el Modelo C4, diagramas UML y el Modelo Entidad-Relación (ERD). Cada uno describe el sistema desde diferentes niveles de abstracción, permitiendo comprender tanto la arquitectura general como los detalles internos de implementación.

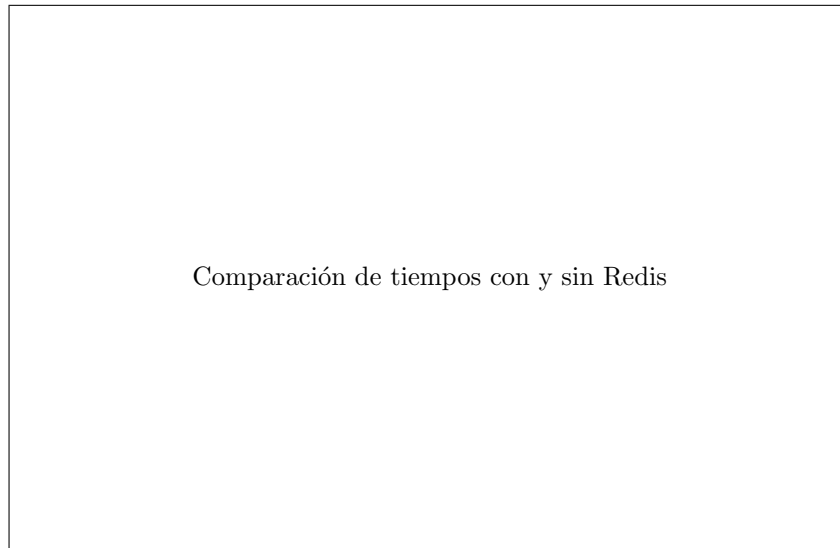


Figure 9: Comparación del rendimiento utilizando PostgreSQL y Redis.

4.1 Modelo C4

El Modelo C4 fue propuesto por Simon Brown como una metodología para documentar arquitecturas de software mediante cuatro niveles de abstracción: Contexto, Contenedores, Componentes y Código. Este modelo facilita la comunicación entre desarrolladores, arquitectos y usuarios al representar progresivamente el nivel de detalle del sistema.

En este proyecto únicamente se desarrollaron los niveles Contexto y Contenedores, conforme a los requerimientos establecidos.

4.1.1 Diagrama de Contexto (Nivel 1)

El diagrama de contexto representa la visión más general del sistema. Su propósito consiste en mostrar cómo el SIGCB-QR interactúa con los actores externos y con los sistemas que participan durante su funcionamiento. Dentro del contexto general intervienen principalmente los siguientes actores:

- Bibliotecario.
- Estudiante.
- Administrador.

Todos ellos interactúan mediante la aplicación web desarrollada en Angular, la cual consume los servicios REST implementados en Spring Boot. El backend administra la lógica de negocio, procesa la autenticación mediante Spring Security, consulta PostgreSQL para la persistencia de datos y utiliza

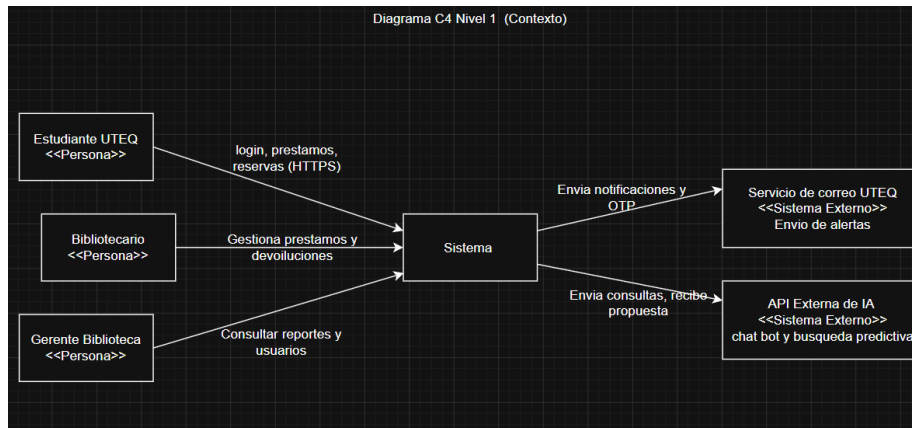


Figure 10: Modelo C4 Nivel 1 (Contexto del Sistema).

Redis para almacenar información temporal y administrar la blacklist de tokens JWT.

Este nivel permite comprender rápidamente el alcance general del sistema y las tecnologías involucradas durante su funcionamiento.

4.1.2 Diagrama de Contenedores (Nivel 2)

El nivel de contenedores describe los principales bloques tecnológicos que conforman el sistema y las relaciones existentes entre ellos.

En el proyecto SIGCB-QR se identifican los siguientes contenedores principales:

- Aplicación Web Angular.
- API REST Spring Boot.
- PostgreSQL.
- Redis.

Angular constituye la interfaz utilizada por los usuarios para administrar libros, préstamos y devoluciones.

Spring Boot implementa toda la lógica de negocio, administra la autenticación mediante JWT, ejecuta las reglas del sistema y coordina el acceso a la base de datos.

PostgreSQL almacena permanentemente toda la información relacionada con usuarios, libros, préstamos, categorías y demás entidades del sistema.

Redis funciona como memoria caché y administra temporalmente la blacklist de identificadores JWT utilizados durante el proceso de cierre de sesión.

La comunicación entre Angular y Spring Boot se realiza mediante servicios REST utilizando el protocolo HTTP, mientras que Spring Boot mantiene conexiones independientes con PostgreSQL y Redis.

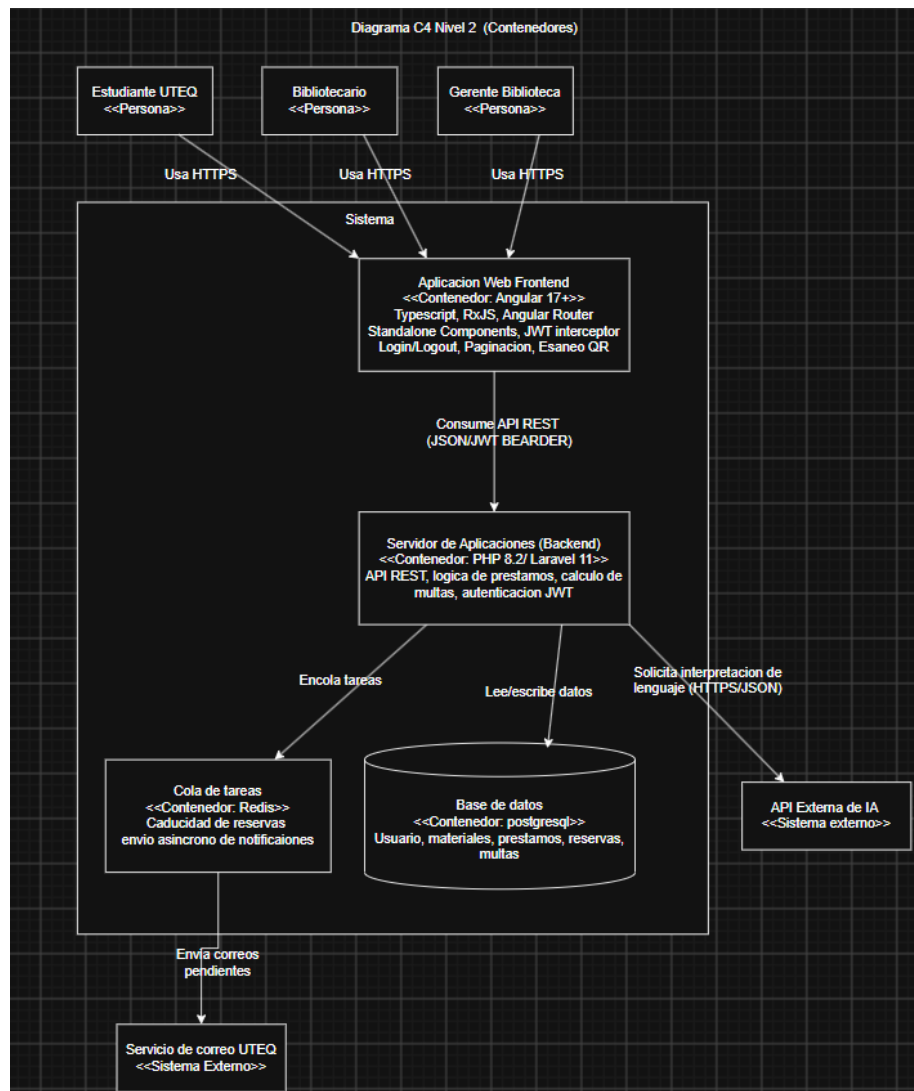


Figure 11: Modelo C4 Nivel 2 (Contenedores).

4.2 Diagramas UML

Unified Modeling Language (UML) constituye el estándar internacional para la representación gráfica de sistemas orientados a objetos. UML facilita la documentación de los distintos componentes del software mediante diversos tipos de diagramas que describen tanto la estructura como el comportamiento del sistema.

Dentro del presente proyecto se desarrollaron los diagramas de Casos de Uso, Clases y Secuencia.

4.2.1 Diagrama de Casos de Uso

El diagrama de casos de uso representa las funcionalidades ofrecidas por el sistema desde la perspectiva del usuario.

Entre las principales funciones implementadas se encuentran:

- Iniciar sesión.
- Registrar usuarios.
- Administrar libros.
- Registrar préstamos.
- Registrar devoluciones.
- Consultar historial.
- Generar códigos QR.
- Administrar roles.

Este diagrama permite identificar claramente los servicios disponibles para cada actor del sistema y facilita la validación de los requerimientos funcionales.

4.2.2 Diagrama de Clases

El diagrama de clases representa la estructura estática del sistema, mostrando las entidades, atributos, métodos y relaciones existentes entre las diferentes clases implementadas.

Dentro del proyecto destacan entidades como:

- Usuario.
- Rol.
- Libro.
- Autor.
- Categoría.

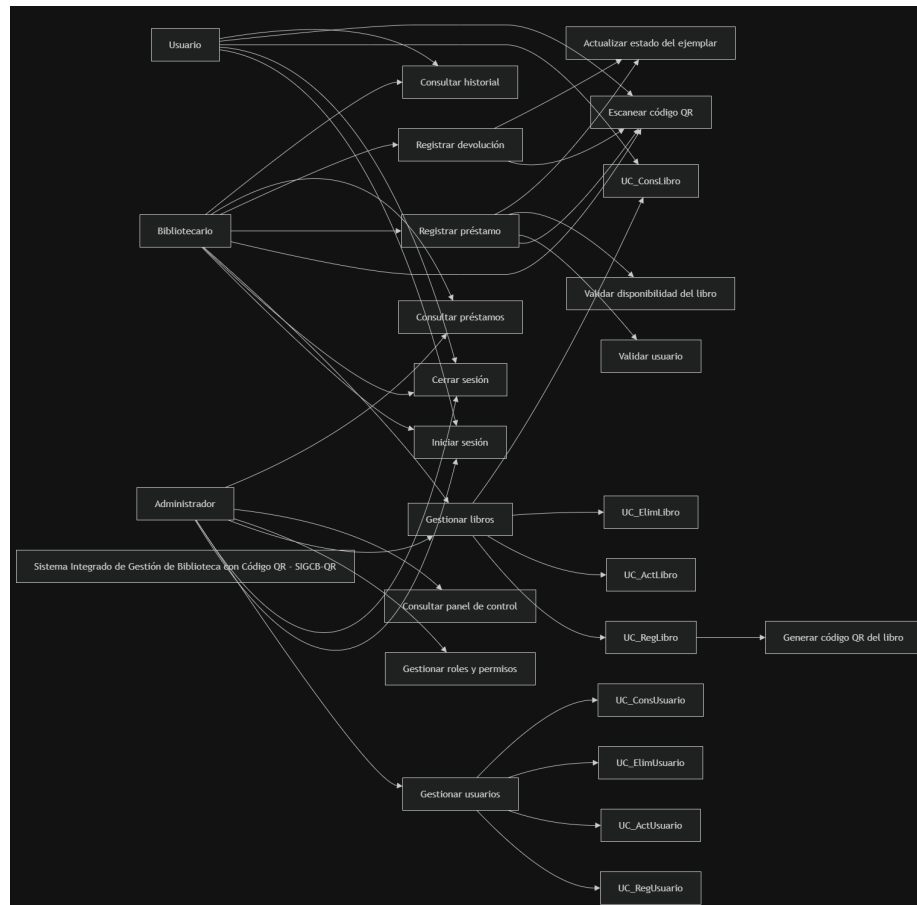


Figure 12: Diagrama de Casos de Uso del Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR).

- Préstamo.
- Devolución.

Asimismo, se representan las relaciones de asociación, composición y multiplicidad necesarias para garantizar la integridad del modelo de dominio.

Este diagrama constituye una guía importante durante la implementación mediante Spring Data JPA, ya que las entidades representadas corresponden directamente con las tablas almacenadas en PostgreSQL.

4.2.3 Diagrama de Secuencia

El diagrama de secuencia describe cronológicamente la interacción entre los diferentes componentes durante una operación específica.

En este proyecto el escenario documentado corresponde al proceso de autenticación mediante JWT.

Durante dicho proceso participan los siguientes elementos:

1. Usuario.
2. Angular.
3. Spring Boot.
4. Spring Security.
5. PostgreSQL.
6. Redis.

El diagrama representa el intercambio de mensajes desde el inicio de sesión hasta la generación del token, incluyendo posteriormente el proceso de validación del JWT y la consulta de la blacklist almacenada en Redis.

Esta representación facilita la comprensión del flujo interno de autenticación y permite identificar claramente las responsabilidades de cada componente.

4.3 Modelo Entidad-Relación (ERD)

El Modelo Entidad-Relación constituye una representación conceptual de la base de datos utilizada por el sistema. Su objetivo consiste en describir las entidades, atributos y relaciones existentes entre los diferentes objetos persistentes.

Dentro del SIGCB-QR el modelo contempla entidades como Usuario, Rol, Libro, Categoría, Autor, Préstamo y Devolución, estableciendo relaciones uno a muchos y muchos a muchos según los requerimientos funcionales definidos durante el análisis del sistema.

Este modelo sirvió como base para la implementación de las entidades JPA y para la creación del esquema físico administrado mediante Flyway en PostgreSQL.

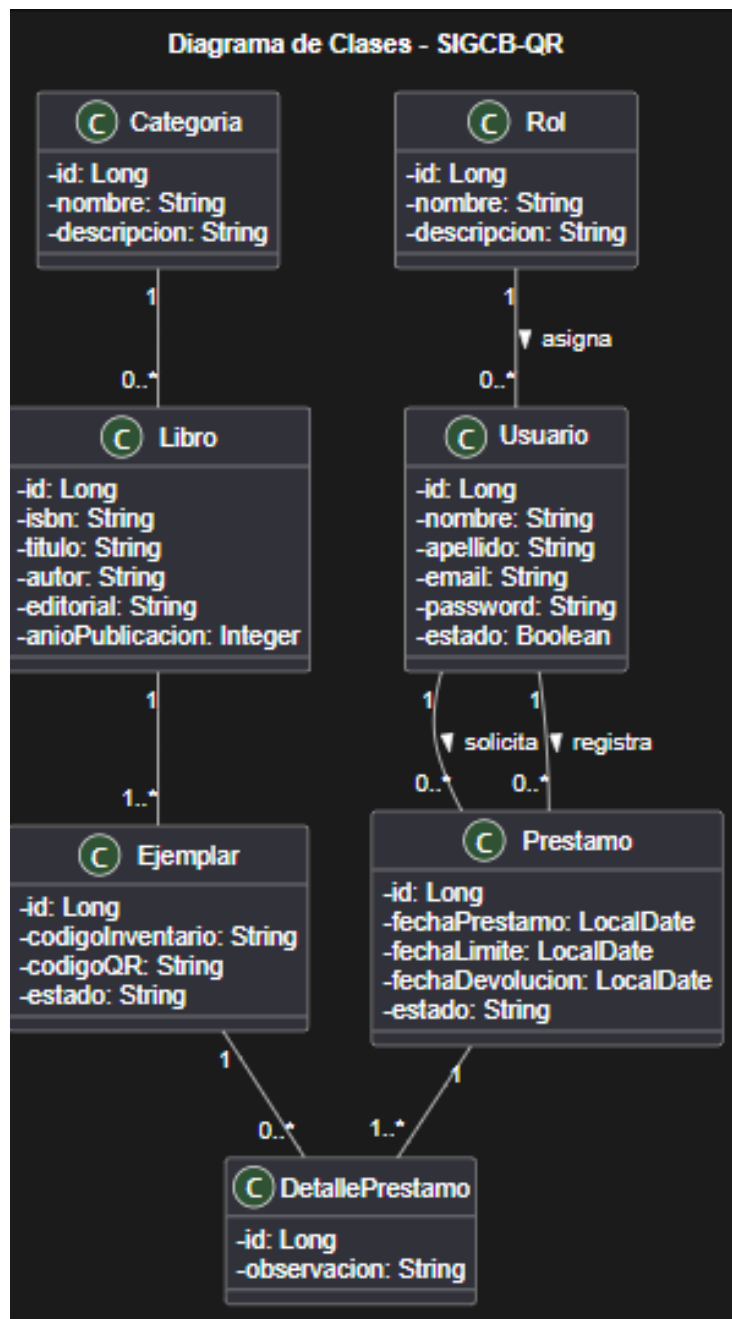


Figure 13: Diagrama de clases del Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR).

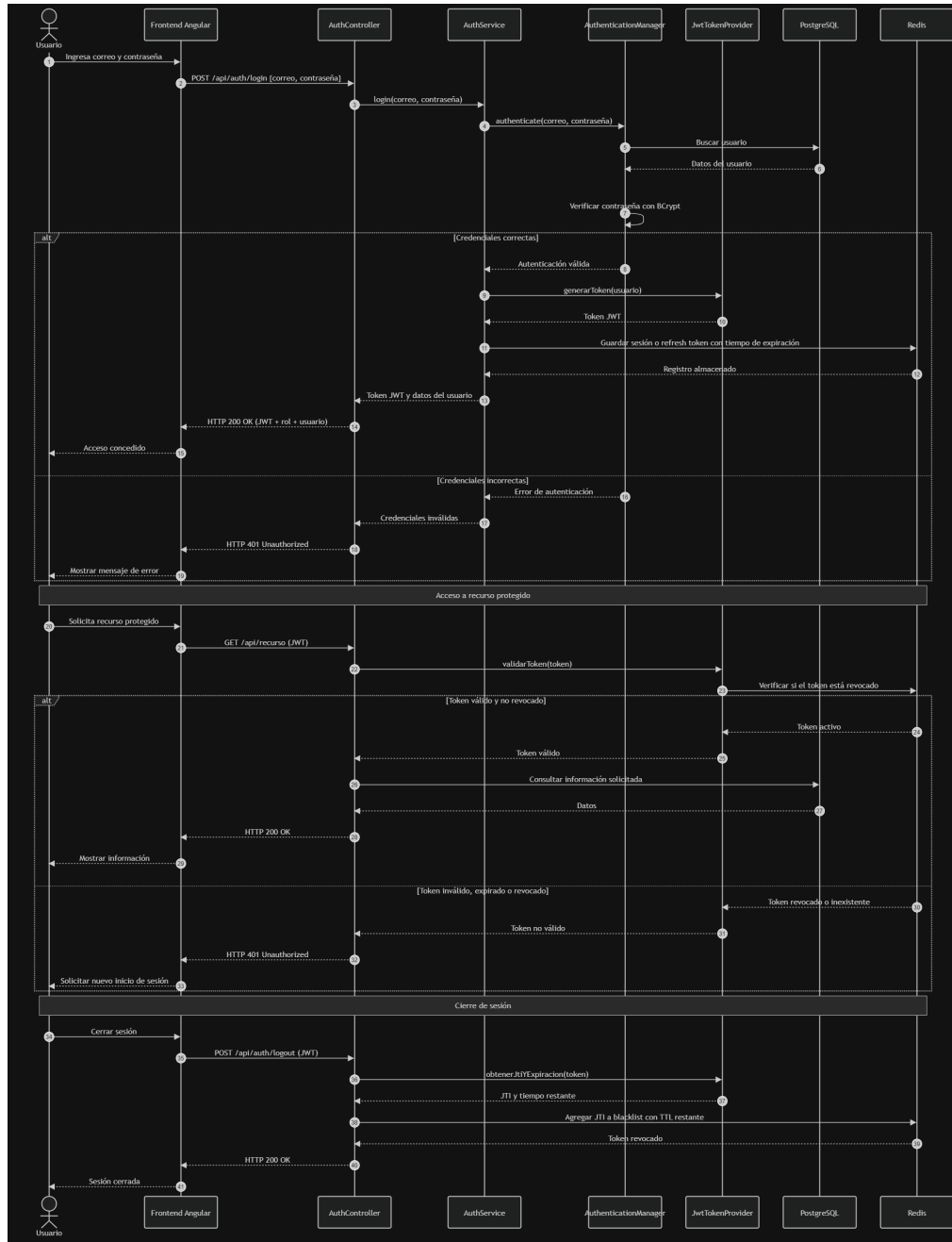


Figure 14: Diagrama de secuencia del proceso de registro de préstamo en el sistema SIGCB-QR.

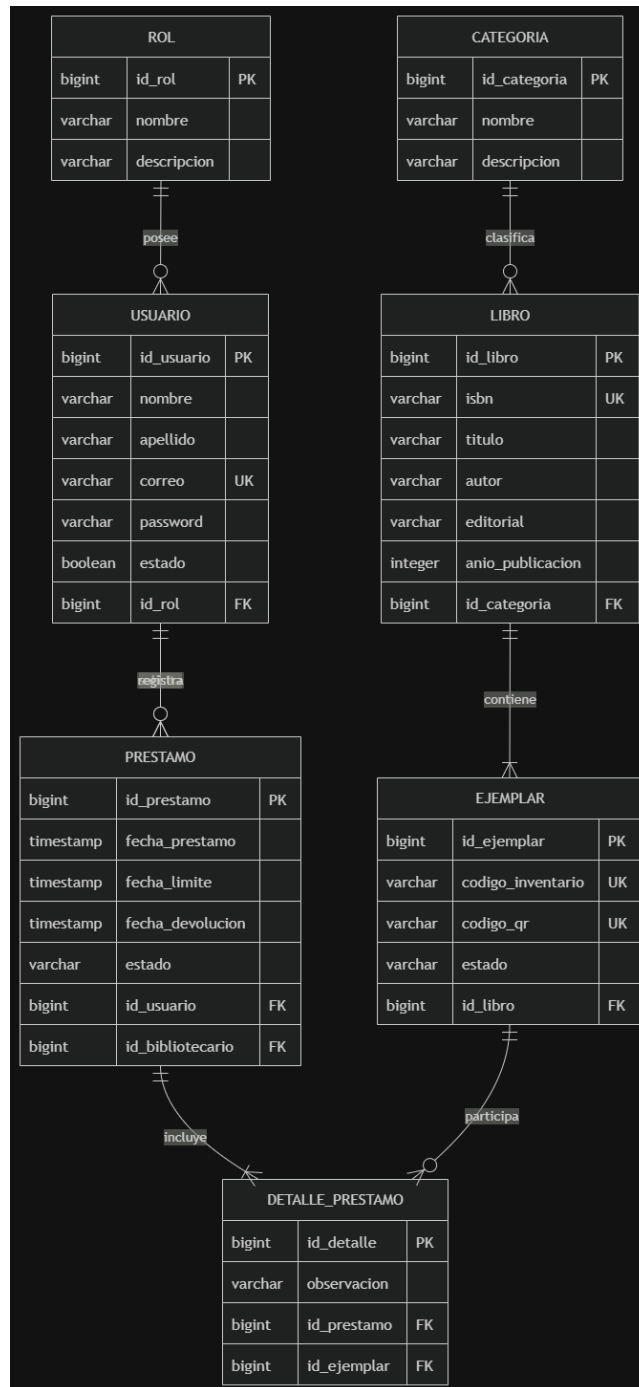


Figure 15: Modelo Entidad-Relación del Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR).

5 Architecture Decision Records (ADR)

Los *Architecture Decision Records* (ADR) constituyen documentos utilizados para registrar las decisiones arquitectónicas tomadas durante el desarrollo de un sistema. Cada ADR describe el problema identificado, las alternativas analizadas, la decisión adoptada y las consecuencias derivadas de dicha elección.

Documentar estas decisiones facilita el mantenimiento del proyecto, mejora la comunicación entre los integrantes del equipo y proporciona una justificación técnica que puede consultarse durante futuras modificaciones del sistema [1]. En el desarrollo del Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR) se documentaron dos decisiones arquitectónicas fundamentales relacionadas con el desarrollo del cliente web y la optimización del rendimiento.

5.1 ADR-002: Selección de Angular como Framework Frontend

5.1.1 Estado

Aceptado.

5.1.2 Contexto

Durante la etapa de diseño fue necesario seleccionar el framework encargado de implementar la interfaz gráfica del sistema. Entre las principales alternativas consideradas se encontraban Angular y React, dos de las tecnologías más utilizadas actualmente para el desarrollo de aplicaciones web. El sistema requería una arquitectura organizada, soporte para componentes reutilizables, integración sencilla con servicios REST y herramientas que facilitaran el desarrollo de aplicaciones empresariales de gran tamaño.

5.1.3 Alternativas Evaluadas

Angular

- Framework completo desarrollado por Google.
- Arquitectura basada en componentes.
- Integración nativa con TypeScript.
- Inyección de dependencias.
- Routing integrado.
- Formularios reactivos.
- Excelente soporte para aplicaciones empresariales.

React

- Biblioteca desarrollada por Meta.
- Mayor flexibilidad.
- Amplio ecosistema.
- Excelente rendimiento.
- Requiere bibliotecas adicionales para routing, formularios y manejo del estado.

5.1.4 Decisión

Se seleccionó Angular debido a que proporciona una solución integral para aplicaciones empresariales, incorporando herramientas oficiales para navegación, formularios, validaciones, comunicación HTTP y organización modular del proyecto.

Asimismo, Angular facilita la integración con Spring Boot mediante servicios REST y proporciona una estructura uniforme que favorece el mantenimiento del código fuente.

5.1.5 Consecuencias

La utilización de Angular permitió:

- Mejor organización del proyecto.
- Desarrollo modular.
- Reutilización de componentes.
- Mayor mantenibilidad.
- Integración sencilla con Spring Boot.
- Compatibilidad con TypeScript.
- Mejor escalabilidad para futuras versiones del sistema.

Table 1: Comparación entre Angular y React

Característica	Angular	React
Framework completo	Sí	No
TypeScript	Nativo	Opcional
Routing	Integrado	Biblioteca externa
Inyección de dependencias	Sí	No
Aplicaciones empresariales	Excelente	Muy buena
Curva de aprendizaje	Alta	Media

5.2 ADR-003: Implementación de Redis

5.2.1 Estado

Aceptado.

5.2.2 Contexto

Durante el desarrollo del backend se identificó que las consultas repetitivas sobre PostgreSQL incrementaban el tiempo de respuesta del sistema. Además, JWT requería un mecanismo que permitiera invalidar inmediatamente los tokens después del proceso de cierre de sesión.

Por esta razón fue necesario incorporar una tecnología especializada para almacenamiento temporal de información.

5.2.3 Alternativas Evaluadas

Sin Caché

- Implementación sencilla.
- Mayor carga sobre PostgreSQL.
- Mayor tiempo de respuesta.

Redis

- Base de datos en memoria.
- Muy alta velocidad.
- TTL automático.
- Integración con Spring Boot.
- Compatible con Cache-Aside.
- Ideal para blacklist JWT.

5.2.4 Decisión

Se decidió utilizar Redis debido a que proporciona almacenamiento completamente en memoria, permitiendo disminuir significativamente los tiempos de respuesta del sistema.

Además, Redis permite administrar automáticamente la expiración de registros mediante TTL, característica especialmente útil para almacenar temporalmente los identificadores JTI utilizados durante el proceso de Logout.

5.2.5 Consecuencias

La incorporación de Redis produjo las siguientes mejoras:

- Disminución del tiempo de respuesta.
- Reducción de consultas sobre PostgreSQL.
- Mayor escalabilidad.
- Implementación segura del Logout.
- Invalidación inmediata del JWT.
- Integración sencilla con Spring Boot.

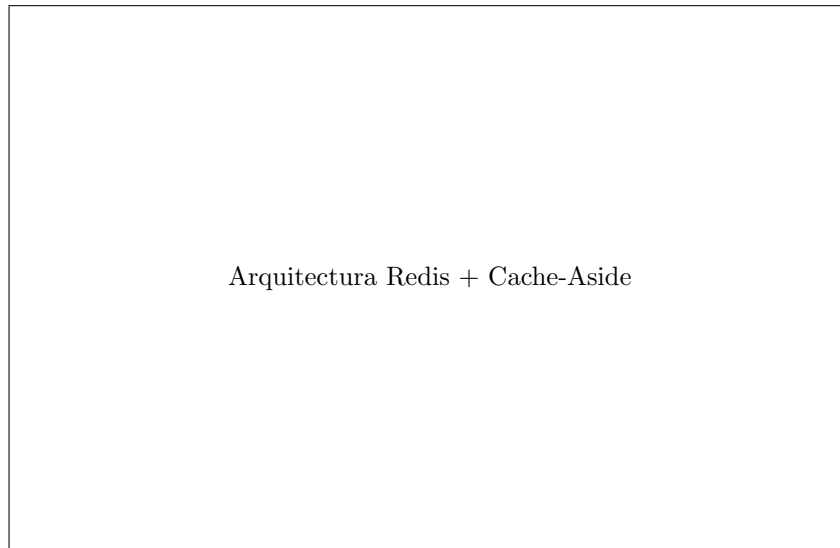


Figure 16: Implementación del patrón Cache-Aside utilizando Redis.

6 Resultados

Una vez implementadas las funcionalidades del Sistema Integrado de Gestión de Biblioteca con Código QR (SIGCB-QR), se realizaron diferentes pruebas funcionales y de rendimiento con el propósito de verificar el correcto funcionamiento de cada uno de los componentes desarrollados.

Las pruebas de autenticación demostraron que el mecanismo Stateless implementado mediante Spring Security y JSON Web Token (JWT) permite validar correctamente la identidad de los usuarios sin necesidad de mantener sesiones almacenadas en el servidor. El uso de cookies configuradas con los

atributos `HttpOnly`, `Secure` y `SameSite` incrementó la seguridad de la aplicación al reducir el riesgo de ataques XSS y CSRF.

Durante las pruebas de cierre de sesión se comprobó que el identificador único (JTI) del token es almacenado correctamente dentro de Redis. Posteriormente, todas las solicitudes realizadas utilizando un token previamente invalidado fueron rechazadas automáticamente por Spring Security, devolviendo el código HTTP 401 (Unauthorized). Este comportamiento demuestra que el mecanismo de blacklist cumple correctamente su objetivo.

Las operaciones CRUD desarrolladas mediante Spring Data JPA funcionaron correctamente sobre todas las entidades del sistema, incluyendo usuarios, libros, autores, categorías y préstamos. Asimismo, la paginación implementada mediante `Pageable` permitió administrar grandes volúmenes de información reduciendo la cantidad de datos transferidos hacia el cliente.

Las migraciones administradas mediante Flyway facilitaron la creación automática del esquema relacional y permitieron incorporar los registros semilla necesarios para las pruebas del sistema, garantizando consistencia entre los diferentes ambientes de desarrollo.

En cuanto al rendimiento, las pruebas realizadas demostraron que Redis disminuye significativamente el tiempo necesario para recuperar información consultada frecuentemente. Después de la primera consulta, los datos permanecen temporalmente almacenados en memoria, evitando nuevos accesos a PostgreSQL y reduciendo considerablemente el tiempo de respuesta.

El cálculo del *Speedup* confirmó que la incorporación del patrón Cache-Aside mejora el desempeño general de la aplicación, especialmente cuando múltiples usuarios realizan consultas repetitivas sobre la misma información.

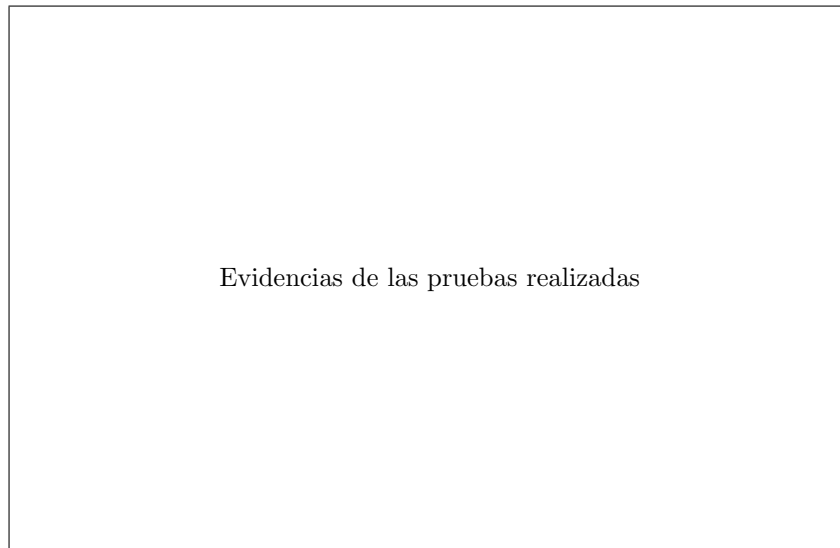


Figure 17: Resultados obtenidos durante las pruebas funcionales del sistema.

7 Conclusiones

El desarrollo del Sistema Integrado de Gestión de Biblioteca con Código QR permitió comprobar la importancia de utilizar arquitecturas modernas para el desarrollo de aplicaciones empresariales. La Arquitectura en Capas facilitó la separación de responsabilidades entre la presentación, la lógica de negocio y la persistencia de datos, obteniendo una aplicación modular, organizada y fácilmente mantenible.

La implementación de autenticación Stateless mediante Spring Security y JSON Web Token eliminó la dependencia de sesiones almacenadas en el servidor, permitiendo construir una solución escalable y compatible con arquitecturas distribuidas. La incorporación de cookies HttpOnly incrementó el nivel de seguridad del sistema al impedir el acceso directo al token desde código JavaScript.

La utilización de Redis como mecanismo de almacenamiento temporal permitió implementar correctamente la invalidación inmediata de JWT mediante una blacklist de identificadores JTI. Además, el patrón Cache-Aside redujo significativamente el número de consultas realizadas sobre PostgreSQL, mejorando el tiempo de respuesta de la aplicación.

Spring Data JPA simplificó considerablemente el desarrollo de la capa de persistencia al automatizar las operaciones CRUD, mientras que Flyway garantizó un adecuado control de versiones sobre el esquema de la base de datos y la carga automática de información inicial.

Finalmente, la documentación arquitectónica desarrollada mediante UML, Modelo C4 y Architecture Decision Records permitió representar claramente la estructura del sistema y justificar técnicamente las principales decisiones tomadas durante su implementación.

8 Recomendaciones

Como trabajo futuro se recomienda incorporar mecanismos de autenticación multifactor (MFA) para incrementar la seguridad del sistema.

Asimismo, resulta conveniente implementar monitoreo mediante Spring Boot Actuator y herramientas como Prometheus y Grafana con el propósito de supervisar el rendimiento de la aplicación en tiempo real.

También se recomienda ampliar el uso de Redis para almacenar información estadística y consultas altamente frecuentes, optimizando aún más el desempeño del sistema.

Finalmente, se sugiere realizar pruebas de carga utilizando herramientas especializadas como Apache JMeter o Gatling para evaluar el comportamiento del sistema bajo condiciones de alta concurrencia.

References

- [1] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [2] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley, 2021.
- [3] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2020.
- [4] C. Walls, *Spring in Action*, 6th ed. Shelter Island, NY, USA: Manning Publications, 2022.
- [5] L. Spilca, *Spring Security in Action*. Shelter Island, NY, USA: Manning Publications, 2020.
- [6] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [7] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2018.
- [8] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [9] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.
- [10] S. Brown, *Software Architecture for Developers*. Leanpub, 2018.
- [11] C. Bauer and G. King, *Java Persistence with Hibernate*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2022.
- [12] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.
- [13] VMware, "Spring Boot Reference Documentation," 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [14] VMware, "Spring Security Reference Documentation," 2024. [Online]. Available: <https://docs.spring.io/spring-security/reference/>
- [15] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/>
- [16] Redis Inc., "Redis Documentation," 2024. [Online]. Available: <https://redis.io/docs/>
- [17] Redgate Software, "Flyway Documentation," 2024. [Online]. Available: <https://documentation.red-gate.com/fd>

- [18] Google, “Angular Documentation,” 2024. [Online]. Available: <https://angular.dev>